OPEN AGENT HACKATHON 2026 · TRACK 2: THE AGENT THAT CAN EXPLAIN WHY

Does "fixed" mean fixed?

A multi-agent assessor that checks a vendor's claim against your own evidence, and says what is still missing.

NVD

CRITICAL, CVSS 10.0

VENDOR CHANGELOG

"Disabled JNDI message lookups by default"

INTERNAL TICKET

2 of 4 clusters unconfirmed

↓

Decision: unresolved

The Revolutionists · github.com/nullstarr/vuln_agent

THE PROBLEM

A fix is announced. Who checks that it reached your systems?

Public database (NVD)

Describes the flaw and rates its severity.

Blind spot: Says nothing about your systems.

Vendor changelog

Announces a fix, often with conditions attached.

Blind spot: Its claim covers the product, not your deployment.

Internal ticket

Records what your team really checked.

Blind spot: Often partial, pending, or silent.

Equifax 2017: attackers used an Apache Struts flaw (CVE-2017-5638) for which a fix had already been published.

WHO IT IS FOR

Analysts who must decide what to patch first



Vulnerability analysts and IT operations leads

Their question, every time a flaw is announced: “Is this really resolved here, and what do I do first?”



A decision with its evidence

Resolved or unresolved, and which source said what.



What would resolve it

The concrete next check, quoted from the text that raised the doubt.



A ranked list across cases

Unresolved, severe or known-exploited first, each with its reason.

Nothing is "resolved" until your own evidence says so.

The vendor's claim is checked against the internal ticket, and the internal check has the last word.



HOW IT WORKS

Four agents, one rule

1

Sources

NVD entry, vendor changelog, internal ticket.

2

SourceMapper

Rates how certain each source is: rules first, optional language model.

3

World Model

Meterless engine keeps facts with provenance and settles conflicts by policy.

4

Reconciler + Challenger

Draft, challenge, revise: never resolved on the vendor's word alone.

5

Reporter

Decision, evidence trail, what would resolve it, triage rank.

Data layer: Zetaris. The three sources are REST tables joined by a SQL view; the tool reads cases with SQL. Files stay as fallback.

Model layer: rule-based by default; Nemotron 3.5 lightning (NVIDIA API) measured; Meterless Runtime provider tested on a stand-in only

PRODUCT: TRIAGE

Which case first? A ranked list, with the reason

#	CVE	Baseline	Decision	Why
1	CVE-2021-44228	CRITICAL 10.0	unresolved	EXPLOITED; internal: unconfirmed item; vendor: carries an exception
2	CVE-2017-5638	CRITICAL 9.8	unresolved	EXPLOITED; internal: unconfirmed item
3	CVE-2014-0160	HIGH 7.5	unresolved	EXPLOITED; internal: work still pending
4	CVE-2024-7002	HIGH 7.5	unresolved	EXPLOITED (simulated feed); vendor: no vendor statement
5	CVE-2024-8888	CRITICAL 9.3	unresolved	internal: unconfirmed item; vendor: carries an exception
9	CVE-2023-0004	HIGH 7.4	unresolved	vendor: claim broader than backed
10	CVE-2022-22965	CRITICAL 9.8	resolved	vendor claim and internal evidence agree

Real output of python3 main.py on the 14 sample cases; ranks 6 to 8 and 11 to 14 not shown. Rule: unresolved and (severe or known to be exploited) first, resolved last. Tickets are synthetic.

PRODUCT: EXPLAIN WHY

Every answer shows its evidence and what is missing

```
CVE-2021-44228   baseline CRITICAL (CVSS 10.0)

Final assessment: CRITICAL - NOT downgraded, internal evidence
insufficient

Confidence: low - unresolved, human review required


--- WHAT WOULD RESOLVE THIS ---

1. Internal: the text itself says something is unconfirmed
('...lookup setting on those two clusters is unconfirmed as of
this ticket.'). Verify that item directly.

2. Vendor: it carries an exception ('Applications that do not
override this system property are not affected by remote lookup
substitution.'). Confirm the exception does not apply to our
systems.
```

Evidence trail

From the Meterless engine: the belief was taken from Ticket OPS-4821; the vendor's changelog entry was superseded.

Policy: internal ticket, then NVD, then vendor.

24 events written to JSONL, each with its source.

PRODUCT: TIMELINE, STRUTS, THE EQUIFAX PATTERN

The picture changes as evidence arrives

Day 0

unresolved

Public database entry appears

internal: no check recorded;
vendor: no vendor statement

Day 1

unresolved

Apache announces fixed versions

internal: no check recorded

Day 4

unresolved

Internal scan: 13 of 14 applications confirmed, the dispute portal is not

internal: unconfirmed item

Names the one application that is not confirmed.

Day 9

resolved

Dispute portal upgraded and re-tested

resolved: vendor fix confirmed by the internal ticket

Only now does the decision change.

Meterless engine: each evidence source existed exactly from its first day (13 of 13 time-travel checks). The NVD record and the Apache advisory are real; the timeline and internal tickets are simulated.

SPONSOR TECHNOLOGY: METERLESS

The World Model engine behind the explanation

0

new events when the same evidence is ingested again

13 / 13

time-travel checks: each source existed exactly from its first day

74

checks in the Meterless test suite

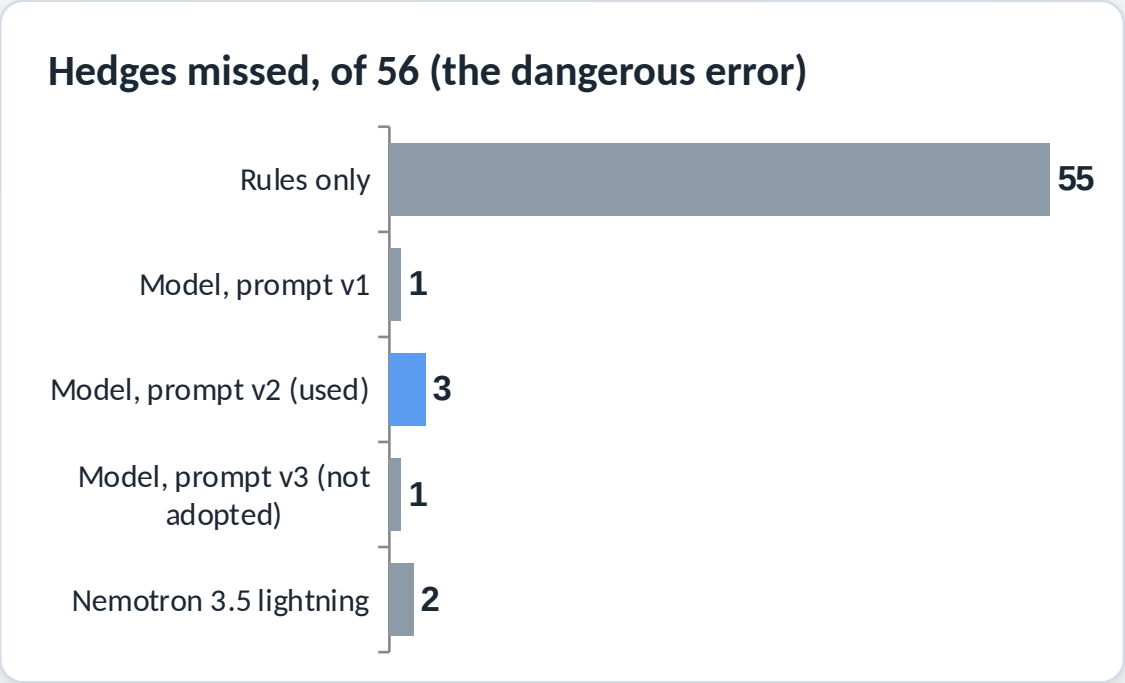
- Only the World Model engine is mounted; H-MEM, Markovian, Scout Intent and Swarm are not.
- The unmodified reference engine, driven from Python; a Python port is the fallback.
- Idempotent ingest, and a crash mid-run resumes cleanly.
- Every event is logged as JSONL with its source.

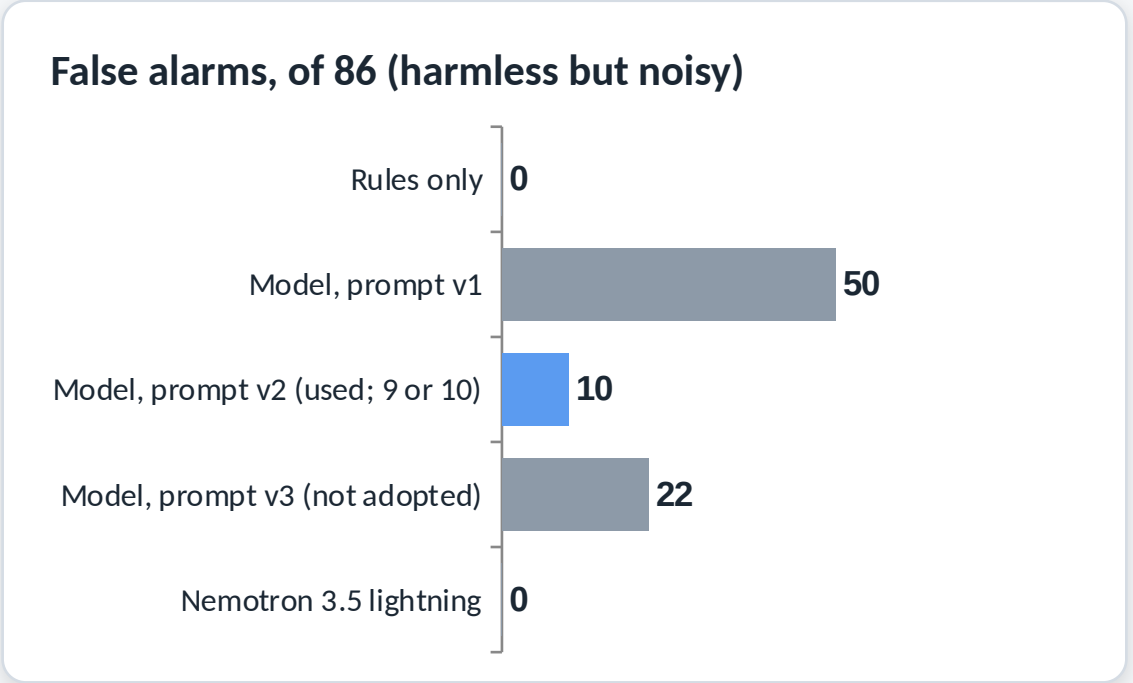
Tested, and two limits found

The reference writes its store file non-atomically, and after a restart it loses the "superseded" markers. We handle both without modifying the engine. The key safeguards were each shown to make a test fail when removed.

MEASURED, NOT CLAIMED

We measured our weak spot, then improved it





Prompt v2 cut false alarms from 50 to 9 or 10 for two more missed hedges; a third prompt did not hold on the test half. Nemotron 3.5 lightning met our pre-set bar on the test half (2 of 56 missed, 0 of 86 false alarms, same in two runs).

Test half of a 286-text corpus we wrote ourselves. On 24 real advisory sentences (author-labelled, one run) Nemotron rated 14 of 17 clear ones uncertain: the result does not carry over. NVIDIA API directly, not Nightshift.

SPONSOR TECHNOLOGY

How each sponsor technology is used

Sponsor	Used for	Status
Meterless	World Model engine: facts with provenance, conflict policy, audit log. Runtime provider built (no Linux Nightshift)	**Engine tested; provider on a stand-in only**
NVIDIA	Nemotron 3.5 lightning judges how certain a statement is; measured on our corpus	*Run live, two runs; via NVIDIA API, not Nightshift*
Zetaris	*Data layer: sources as REST tables, SQL view, the tool reads cases with SQL*	*Integrated; checked live on 17 cases*
Cursor	*Brainstorming only; the code was written with Claude*	*Optional under the final rules*

What is unique: the engine settles our core problem, so its record is the explanation; Zetaris serves the evidence with SQL; every claim here is backed by a test or a measured run, and the limits are stated.

HONEST LIMITS AND NEXT STEPS

What it cannot do yet, and what comes next

Limits

- Tickets and timelines are simulated; the corpus is our own, not independent data.
- Misses confident claims wider than what is backed. Nemotron: good on our corpus, noisy on 24 real sentences (14/17 false alarms).
- Meterless Runtime provider never ran against a real Nightshift.

Next

- Run the provider against a real Nightshift (Windows, Mac or DGX).
- Repeat the Nemotron measurement on independent sentences.
- Add asset value and exposure to the ranking (exploitation flag: done).
- Use Meterless memory so a case carries its history.

Explain why, with evidence you can check.

`github.com/nullstarr/vuln_agent`

Track 2: The Agent That Can Explain Why · The Revolutionists